

Sarvah: Backend Functionality Explanation

Detailed Overview for Jury Questions & Future Scope

1. Fully Built Functionalities (How They Work at Backend)

A. Authentication & Session Management

File: `lib/auth.ts`

- **Registration/Login Flow** (`/api/auth/register`, `/api/auth/login`):
 1. Client sends credentials (name, phone, password)
 2. Server validates credentials and hashes password using Node's native `crypto.scryptSync` (demonstrates security awareness)
 3. User saved to file-based data store (`data/users.runtime.json`)
 4. Server signs a JWT using the `jose` library (edge-compatible, HS256 algorithm)
 5. JWT set as `HttpOnly` secure cookie via `createSession(user)`
 6. Cookie expires in 7 days
- **Session Handling:**
 - Layout and dashboard automatically query `/api/auth/me` to fetch user details
 - Middleware (`middleware.ts`) protects routes based on session validity
 - `getSession()` decrypts the cookie and returns user payload
 - `deleteSession()` clears the cookie on logout
- **Security:** Stateless authentication - the signed HTTP-only cookie contains all necessary information. No need to constantly query database to check login status. `jose` library is designed to run on Vercel Edge functions for lightning-fast middleware route protection.

B. Smart Decision Engine (SELL/WAIT Recommendations)

File: `lib/recommendation/engine.ts`

- **Rules-based engine** that analyzes 60 days of historical Mandi price data
- **Algorithm Flow:**
 1. Input: crop, district, 60-day price history, storage availability, cash urgency
 2. If < 8 data points: return HOLD with 0.3 confidence
 3. Calculate **percentile** of current price against sorted 60-day history
 4. Calculate **7-day trend**: compare today's price vs price 7 days ago
 - up if > 2% increase, down if > 2% decrease, flat otherwise
 5. Determine **seasonality hint** based on crop and month
 - Onion: prices spike in Aug-Sep (lean supply)
 - Tur: prices drop in Oct-Nov (harvest season)
 - Soybean: prices firm up in Sep-Oct (supply tightens)
- **Decision Rules** (priority order):
 - percentile >= 80 && trend7d !== "up" → **SELL_NOW** (0.85 confidence)
 - percentile >= 80 && trend7d === "up" → **WAIT_3_DAYS** (0.7 confidence)
 - percentile <= 30 && trend7d === "down" → **WAIT_2_WEEKS** (0.65 confidence)
 - percentile <= 30 && trend7d === "up" → **WAIT_3_DAYS** (0.7 confidence)
 - trend7d === "down" && percentile >= 50 → **SELL_NOW** (0.7 confidence)
 - Otherwise → **HOLD** (0.5 confidence)
- **Constraint Adjustments:**
 - **Storage constraint:** If farmer has no storage and recommendation is WAIT, suggest SELL_NOW to avoid moisture/pest loss
 - **Liquidity constraint:** If farmer needs cash immediately, force SELL_NOW regardless
- **Output:** Returns action, confidence (0-1), English reasoning, Marathi reasoning, current price, percentile, 7-day trend, seasonality hint, and storage warning flag

C. FPO Pooling & Premium Calculation

File: `lib/data/store.ts` (createLot function, lines 86-103)

- When a lot is created via `POST /api/lots` and marked `isFpoPool: true`:
 1. Backend automatically adds **+3.5% premium** to the asking price
 2. `askingPricePerQuintal = Math.round(data.askingPricePerQuintal * 1.035)`
 3. This proves structurally that the system incentivizes farmers to pool yields for better bulk leverage
 4. The premium is reflected in the lot display and offer process

D. Hyperlocal Freight & Distance Deduction

File: `lib/distance.ts`

- **District-to-Distance Matrix** (pre-calculated distances between 5 Maharashtra mandis):

```
Latur: { Latur: 0, Pune: 180, Nashik: 250, Solapur: 150, Nagpur: 350 }
Pune: { Latur: 180, Pune: 0, Nashik: 150, Solapur: 200, Nagpur: 300 }
Nashik: { Latur: 250, Pune: 150, Nashik: 0, Solapur: 250, Nagpur: 400 }
Solapur: { Latur: 150, Pune: 200, Nashik: 250, Solapur: 0, Nagpur: 350 }
Nagpur: { Latur: 350, Pune: 300, Nashik: 400, Solapur: 350, Nagpur: 0 }
```

- **Freight Heuristic:** ₹0.8 per km per quintal (average Maharashtra freight rate)
- **When a buyer views a lot:**
 1. Server retrieves pre-calculated distance (e.g., Latur farmer → Pune buyer = 180km)
 2. Transport cost = km * 0.8 (e.g., 180 * 0.8 = ₹144 per quintal)
 3. **Net Realization Price** = Mandi Price - Transport Cost
 4. Display warning if distance > 50km: ⚠ Far (~350 km) — freight ~₹280/q
- `getDistancesFromFarmer()`: Returns distance info for all mandis from farmer's district, enabling comparison shopping.

E. Transaction Flow (Lot Creation → Offer → Accept → Transaction)

File: `lib/data/store.ts` (`acceptOffer` function, lines 129-171)

1. **Create Offer** (POST `/api/offers`):
 - Buyer submits price per quintal and quantity
 - Offer stored with status "pending"
2. **Accept Offer** (POST `/api/offers/[id]/accept`):
 - Validate offer exists and not already accepted
 - Update offer status to "accepted"
 - Update lot status to "closed"
 - **Create transaction record** with:
 - `lotId, offerId, buyerId, farmerId`
 - `finalPricePerQuintal` (the offer price)
 - `qtyTons` (from offer)
 - `totalAmount = pricePerQuintal * qtyTons * 10` (1 ton = 10 quintals)
 - `closedAt timestamp`
 - Return `{ offer, transaction, lot }` for frontend confirmation
3. **Atomic Updates:** All operations use read-modify-write pattern on JSON files, ensuring data consistency within the prototype scope.

F Demand Post System (Buyer Requests)

File: `lib/data/store.ts` (`createDemand`, `getMatchingDemands`)

- Buyers can post demand requests specifying:
 - Crop type, district, quantity (tons), price range (min/max per quintal)
 - Grade requirement (A/B/C), delivery window (days)
- `getMatchingDemands(crop, district)`: Returns top 3 matching demands for farmer dashboard
- Demand posts stored in `data/demand-posts.runtime.json`

G. Marathi-First Internationalization

File: `lib/i18n.ts` + `messages/en.json`, `messages/mr.json`

- `next-intl` configured with English + Marathi locales
- Marathi-first approach for Maharashtra farmers
- All UI strings mapped to JSON dictionaries
- Dynamic routing: `/en/farmer` vs `/mr/farmer` with persistent language context
- Server Components fetch data on server, reducing client-side JS for faster loading in rural areas

H. Price Trend Visualization

File: `components/charts/PriceTrendChart.tsx` + `recharts`

- Uses Recharts library for declarative SVG charts
- 30-day Mandi Price Trend chart on farmer dashboard
- Fetches price history from seed data via API route `/api/prices`
- Chart integrates natively with React, performs well on mobile devices
- Displayed alongside SELL/WAIT recommendation from the rules engine

2. Partial / Mocked Functionalities

A. Real-Time Mandi API (AGMARKNET)

Current State: Simulated using seeded JSON data (`seed-loader.ts`)

- **How it works:** `lib/data/agmarknet.ts` and `seed-loader.ts` provide mock price data
- **Seed data:** 900+ price points across 60 days for 3 crops (soybean, onion, tur) across 5 districts
- **Guaranteed performance:** Demo works perfectly without relying on unreliable third-party government APIs during pitch

How It Should Work (Future Scope):

- CRON job (serverless worker) pings actual Gov AGMARKNET SOAP/REST endpoints every morning at 9 AM
- Sanitize XML/JSON response and normalize data structure
- Push normalized daily prices into PostgreSQL database

- Real-time price updates would replace the static seed data
- Backend data access layer designed for easy migration (types strictly defined, see `tech_stack_rationale.md` "Future Migration Path")

B. Quality Grading

Current State: Self-declared FAQ (Fair Average Quality) grades A, B, C

- Farmers manually select grade when creating a lot
- Grade stored in lot data and displayed to buyers

How It Should Work (Future Scope):

- Integration with AI-based crop assaying APIs (like AgNext or e-NAM assaying labs)
- Farmers upload photos of their crop
- Computer vision model assesses moisture content and physical damage
- Auto-assigns verifiable grade with digital certification
- Graded produce commands premium pricing in marketplace
- Would require mobile app integration and API gateway setup

C. Verified Buyer Badges

Current State: Mocked trust scores and criteria notes

- Buyer profile shows trustScore (0-100, mocked) and criteriaNotes for tooltips
- Basic badge display without actual verification backend

How It Should Work (Future Scope):

- KYC integration with government IDs (Aadhaar, PAN)
- Bank account verification via micro-deposits
- Blockchain-verified transaction history for each buyer
- Real-time verification status API
- Display badges next to buyer names in marketplace

D. Distance Heuristic (Prototype Scope)

Current State: Partially built with hardcoded distance matrix

- District-to-distance matrix works for 5 Maharashtra mandis
- Freight calculation functional
- Warning trigger at >50km

How It Should Work (Future Scope):

- Integrate with Google Maps Distance Matrix API or OpenRouteService
- Real-time traffic-aware routing
- Per-quintal freight calculation based on actual commodity type
- Seasonal road condition adjustments
- Distance would dynamically calculate for any pincode combination, not just the 5 predefined mandis

3. Deferred Functionalities (Phase 2 Roadmap)

A. Digital Payment Escrow & Smart Contracts

The Plan:

- Integrate payment gateway (Razorpay Route or banking nodal account)
- Act as trusted third-party holding funds

The Flow :

1. When buyer accepts offer, they transfer funds to escrow
2. Funds are locked in escrow account
3. Logistics partner marks lot as "Delivered"
4. Buyer signs off on quality (weighbridge receipt + quality assay)
5. Escrow automatically releases funds to farmer's bank account via UPI/NEFT
6. If dispute arises, funds remain frozen until resolution

Jury Answer Prep: "We've designed the escrow to protect both parties - buyer gets quality assurance, farmer gets guaranteed payment. Integration with Razorpay Route would take ~2 weeks in Phase 2."

B. Logistics Coordination (Kisan Rath Integration)

The Plan:

- Open API bridge to national transport aggregators or local truck unions
- Effectively Uber-ize the farm-gate pickup

The Flow :

1. When transaction is finalized, system broadcasts "Load Request"
2. Local truckers can bid for the delivery route
3. Best bid accepted, truck assigned
4. GPS tracking enabled for shipment
5. Proof of delivery (photograph + weighbridge) uploaded

Jury Answer Prep: "We've identified the logistics gap and have Kisan Rath API integration roadmap. This would complete the full-circle marketplace experience."

C. Dispute Resolution Mechanism

The Plan:

- Formal mediation workflow for quality/quantity disputes

The Flow :

1. If buyer claims Grade C instead of Grade A upon arrival
2. Payment remains frozen in escrow
3. APMC-licensed arbitrator digitally assigned
4. Arbitrator reviews weighbridge and assaying receipts
5. Final binding decision made
6. Escrow releases funds based on arbitrator's decision

Jury Answer Prep: "Disputes are handled through formal mediation with APMC-licensed arbitrators. This builds trust in the marketplace for both farmers and buyers."

4. Backend Technology Summary

All backend functionalities are implemented in **TypeScript** within a **Next.js 14 (App Router)** framework. Key backend files:

Component	File	Lines of Code
Authentication	lib/auth.ts	44
Data Store (CRUD)	lib/data/store.ts	211
Distance/Freight	lib/distance.ts	35
Recommendation Engine	lib/recommendation/engine.ts	191
Types & Interfaces	lib/types.ts	110
i18n Configuration	lib/i18n.ts	~50

Data persistence uses **JSON file-based storage** (`data/*.runtime.json`) with `seed` data for prototype demo. The architecture is designed for easy migration to PostgreSQL + Prisma/Drizzle ORM in Phase 2.

5. Future Migration Path Notes

As documented in `tech_stack_rationale.md`:

- JSON store strictly typed → replacing with Prisma ORM takes < 1 day
- No frontend UI code touched during migration
- All TypeScript interfaces (`types.ts`) remain valid
- API routes remain compatible with minimal adjustments

This comprehensive backend implementation covers 8/18 spec items as fully built, 5/18 as partial/mockred, and 5/18 as deferred for Phase 2, with 100% acknowledgment of all specification items.